SHENZHEN UNIVERSITY

COURSE

Data Analytics in Marketing
Shenzhen University

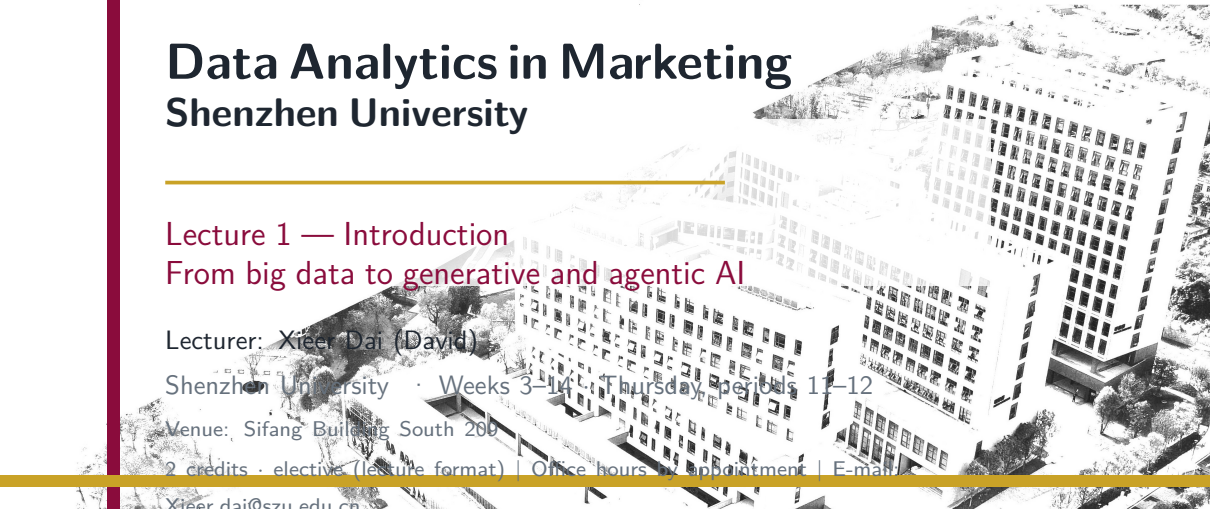
Lecture 1 — Introduction
From big data to generative and agentic AI

Lecturer: Xieer Dai (David)

Shenzhen University · Weeks 3–14 · Thursday, periods 11–12

Venue: Sifang Building South 209

2 credits · elective (lecture format) | Office hours: by appointment | E-mail: Xieer.dai@szu.edu.cn



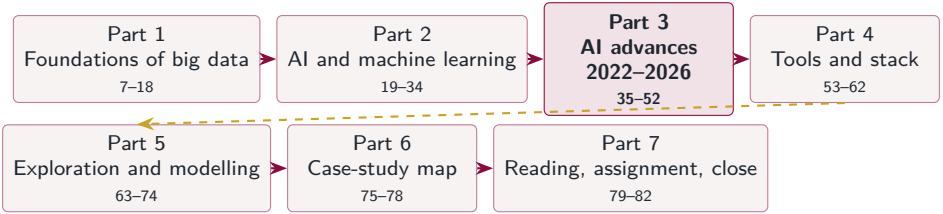
Learning Objectives

After this lecture you should be able to:

1. define big data and its 5 V characteristics, and distinguish structured, semi-structured and unstructured data;
2. explain how the classic analysis workflow (CRISP-DM) maps onto modern MLOps / LLMOps practice;
3. describe any analytics task with the "Machine Learning 1-2-3" frame (data $\rightarrow$ model $\rightarrow$ optimisation);
4. explain the basic principles behind foundation models, reasoning models, agentic AI and RAG;
5. judge where generative AI does and does not fit: hallucination, evaluation, cost and compliance risk;
6. complete an end-to-end data exploration on your own, declaring any AI tools you used.

Source: Course team material; terminology and developments current to 2026 – see the footnote on each slide.

Roadmap



Spine: data → features → model → evaluation → deployment

New this year: GenAI / reasoning models / agentic AI / regulation and compute

Course Guideline

Course: Data Analytics in Marketing (Shenzhen University)
Lecturer: Xieer Dai (David) **E-mail:** Xieer.dai@szu.edu.cn
Weeks: 3–14 **Time:** Thursday, periods 11–12
Venue: Sifang Building South 209
Office hours: by appointment **Office:** Mingde Building 1310
Credits: 2 **Type:** elective, lecture format

How this course treats AI tools

Large language models are welcome as study and coding assistants, but you must (1) declare the tools and prompts you used, (2) verify model output yourself, and (3) never submit unverified model-generated content.

Prerequisites & Reading

Prerequisites

- Programming: any one of C/C++/Java/Python/Matlab (Python is the working language); using an LLM as a coding assistant is encouraged;
- Mathematics: elementary algebra, linear algebra, linear systems, calculus, basic optimisation, probability;
- Statistics: descriptive and inferential statistics, hypothesis testing, estimators and goodness of fit.

Textbooks

- No set textbook; each lecture lists references;
- McKinney (2018) *Python for Data Analysis*;
- Géron (2022) *Hands-On Machine Learning*;
- Chen Qiang (2021) *Machine Learning and Python Applications*.

Supplementary reports 2025–2026

- Stanford HAI, *AI Index Report 2026*;
- McKinsey, *The State of AI*;
- KPMG, *AI Pulse* (enterprise agents).

Assessment, Assignment and AI Rules

Assessment: 10% presentation + 90% project (three of four analytics projects, teams of up to two). The report covers steps, methods, code, results and interpretation.

AI use declaration (with every submission)

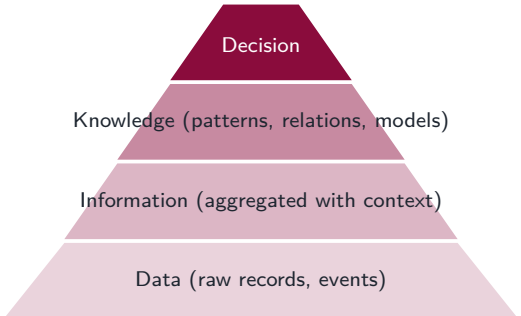
- Which tools (model name + version + date);
- For what: code, prose, analysis, figures;
- The key prompts (keep a prompt log);
- How you verified the output and what you changed.

Not allowed

- Submitting unverified model output;
- Fabricating data, results or references;
- Undeclared ghost-writing.

Source: Course team rules (2026); stricter university rules prevail.

From Data to Decisions: the DIKW Pyramid



The value of analytics lies in turning raw records into executable decisions layer by layer; each layer carries its own methods and quality requirements.

Value comes from the transformation chain, not from data volume itself.

Big Data: the 5 V

Characteristic	Meaning, seen from 2026
Volume	From terabytes to zettabytes; the corpus and log volume behind a single training run far exceeds traditional database design targets
Velocity	Streaming data (clickstreams, payments, sensors, logs); the real-time decision window has shrunk from hours to milliseconds
Variety	Structured tables, text, images, audio and video, graph data, time-series signals
Veracity	Noise, missingness, labelling bias and the authenticity risk introduced by synthetic data
Value	Only data that enters a decision path creates value (analysis $\rightarrow$ product $\rightarrow$ revenue)

Source: Classic 3 V framework (volume/variety/velocity) plus later extensions; modality and governance points drawn from recent practice.

Data Modalities

Structured

Tables and relational databases; typed columns; directly computable.

Semi-structured

JSON / XML / logs / web pages; self-describing but with no fixed schema.

Unstructured

Text, images, audio, video; must first be turned into vectors or features before modelling.

Graph and spatio-temporal

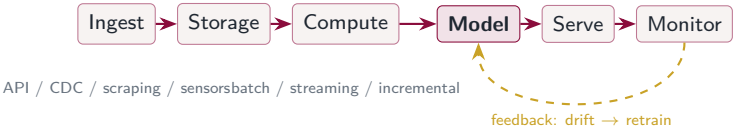
Relationship networks (transfers, social ties), trajectories and geographic rasters.

How it is handled now

- Classic route: cleaning + feature engineering;
- Representation learning: encode unstructured input as embedding vectors;
- Multimodal models: one shared representation for text, image and audio.

Most big-data work is spent turning unstructured inputs into a usable numeric representation.

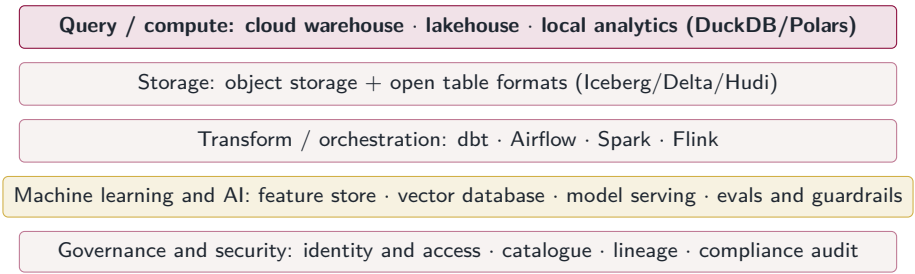
The Data Pipeline



Engineering requirements: idempotent reruns, observability, schema contracts, lineage.

Without monitoring and feedback, models silently decay as the world changes.

The Modern Data Stack, 2026



The trend: compute decoupled from storage, open table formats to avoid lock-in, and the AI layer wired directly into the governed data layer (shared features, RAG).

The AI layer now sits on top of the same governed data platform as BI.

Four Levels of Analytics (plus Generative and Agentic)

Level	Question answered	Typical techniques
Descriptive	What happened?	Reports, dashboards, aggregate statistics
Diagnostic	Why did it happen?	Drill-down, correlation, causal inference
Predictive	What will happen?	Regression, classification, forecasting
Prescriptive	What should we do?	Optimisation, simulation, A/B tests, decision rules
Generative	Can it produce the content or answer directly?	LLMs, multimodal generation, RAG
Agentic	Can it complete a multi-step task on its own?	Planning + tool use + memory + reflection

The classic four levels deliver understanding and decisions; generative and agentic systems add execution.

Analysis Workflow: CRISP-DM versus MLOps / LLMOps

Stage	CRISP-DM (classic)	MLOps / LLMOps (modern)
Understand the problem	Business understanding, define goals	Define metrics + success criteria + risk register
Understand the data	Data understanding, initial exploration	Data catalogue, lineage, quality monitoring
Prepare the data	Cleaning, feature construction	Feature platform, rerunnable pipelines, data contracts
Model	Choose algorithms, train	Experiment tracking, versioning, automated tuning
Evaluate	Offline metrics	Offline + online (A/B, shadow) + cost and latency
Deploy	Report / hand over	CI/CD release, canary, rollback, guardrails
Monitor	Rarely done	Drift detection, prompt versioning, human review loop

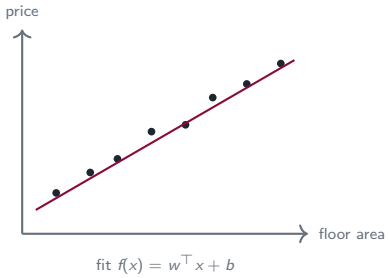
Source: Compiled from the CRISP-DM process and MLOps / LLMOps practice.

Worked Example A: Why House Prices Are a Regression Problem

Task: predict a continuous price $y \in \mathbb{R}$ from house features x.

- Target is continuous $\rightarrow$ regression;
- Usual loss: $\ell(y, f(x)) = (y - f(x))^2$;
- Usual metrics: RMSE, MAE, R^2;
- Typical models: linear regression $\rightarrow$ regularised regression $\rightarrow$ gradient-boosted trees $\rightarrow$ neural networks.

The key is to settle what is being predicted (the type of the target) before choosing a loss and a metric.



Worked Example B: Why Spam Detection is a Classification Problem

Task: decide whether an e-mail is spam, $y \in \{0, 1\}$.

- Target is discrete $\rightarrow$ classification;
- Features: keyword counts, sender attributes, text embeddings;
- Loss: 0–1 loss (not optimisable) $\rightarrow$ cross-entropy (optimisable);
- Metrics: precision, recall, F_1, ROC-AUC (accuracy alone is not enough).

With imbalanced classes, accuracy badly overstates how good the model really is.

Predicted \ actual	$y = 1$ (spam)	$y = 0$ (ham)
$\hat{y} = 1$	TP	FP
$\hat{y} = 0$	FN	TN

$$\text{Precision} = \frac{TP}{TP+FP}, \text{ recall} = \frac{TP}{TP+FN}.$$

Data Quality & Bias (data work is 60–80% of the effort)

Common quality problems

- Missing values (random or systematic);
- Outliers and keying errors;
- Duplicates and inconsistent entities;
- Inconsistent units, timestamps, time zones.

Common sources of bias

- Selection bias (only part of the users);
- Label noise, annotator disagreement;
- Historical prejudice amplified by the model;
- Drift: the training world $\neq$ deployment world.

Practical advice

Profile the data before modelling: missing rates, value ranges, time coverage, group distributions. Put data-quality metrics on the monitoring dashboard alongside model metrics.

Source: Checklist compiled from data-engineering and fairness research practice.

Seven Common Pitfalls

1. Leakage: using future information, or a proxy for the target, as a feature;
2. Metric mismatch: optimising a convenient metric rather than the outcome the business cares about;
3. Correlation as causation: acting on a correlational finding as if it were an intervention;
4. Accuracy only: ignoring class imbalance, asymmetric costs and confidence intervals;
5. Over-tuning: repeatedly tuning on the test set $\rightarrow$ hidden overfitting;
6. Ignoring cost and latency: accuracy improves by 1% while cost rises tenfold;
7. No monitoring or rollback: model disappears after launch and nobody owns the drift.

Rule: build a trustworthy baseline before reaching for a complex model.

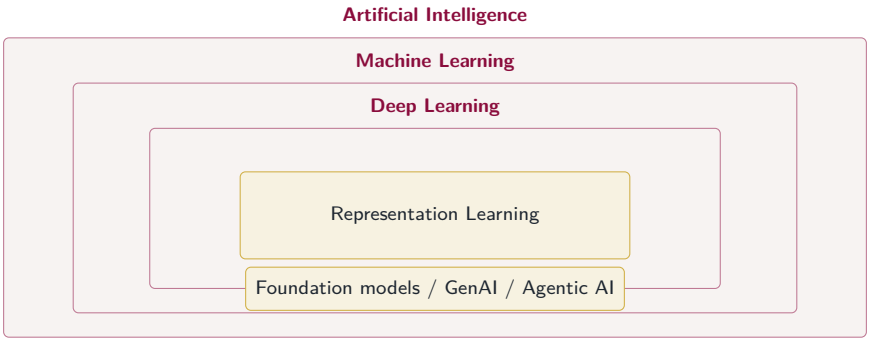
Part 1 Summary

- Big data = volume + velocity + variety + veracity + value, and the core is value entering a decision path;
- The pipeline and the modern data stack decide whether analysis can scale and be reproduced;
- Analytics moves from understanding (descriptive, diagnostic) to prediction and prescription, and now to generative and agentic systems;
- The classic process (CRISP-DM) has to be mapped onto modern engineering practice (MLOps / LLMOps);
- Data quality and bias are first-order problems and take up most of the effort.

Self-check

(1) Is the target continuous or discrete? (2) Do the loss and metric match the business problem? (3) Is there any leakage? (4) What is the data quality? (5) How will it be monitored after launch?

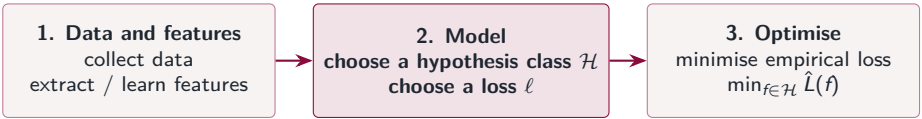
The AI Landscape, 2026



Classic machine learning was largely hand-crafted features plus shallow models; deep learning folded feature learning into the model; foundation models made pretrain-then-adapt the new paradigm.

Generative AI and agentic systems sit inside this landscape, not beside it.

Machine Learning 1-2-3



$$\hat{L}(f) = \frac{1}{n} \sum_{i=1}^{n} \ell\big(f(x_i), y_i\big), \qquad f^* = \arg\min_{f \in \mathcal{H}} \ \hat{L}(f).$$

Step 2 injects prior knowledge (model form and loss); step 3 is pure optimisation; step 1 sets the ceiling.

Model class + loss encode prior knowledge; data and features bound achievable performance.

Learning Paradigms

Paradigm	Form of the data	Typical tasks
Supervised	Labelled (x_i, y_i)	Regression, classification, ranking, object detection
Unsupervised	x_i only	Clustering, dimensionality reduction, anomaly detection, topic models
Self-supervised	Labels constructed from the data itself	Language modelling, masked reconstruction, contrastive learning (LLM pretraining)
Reinforcement learning	Interaction with an environment; rewards	Games, robotics, recommendation, alignment (RLHF/RLVR)

Today's large models owe their capability mainly to self-supervised pretraining plus post-training alignment (human feedback and verifiable rewards).

Source: Paradigm taxonomy follows the classic machine-learning framework; post-training is covered on slide 29.

Train / Validation / Test

- Training set: fit the parameters;
- Validation set: select the model, tune hyper-parameters, early stopping;
- Test set: only for the final report, never reused;
- Split time-series and grouped data by time or entity to avoid leakage.

Use k-fold cross-validation on small samples; stratify when classes are imbalanced.

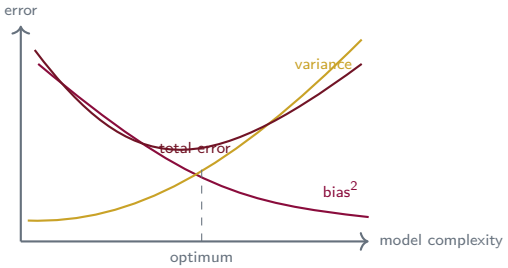
Signals of overfitting

- Training error far below validation error;
- Validation error worsens after model selection;
- Coefficients or features grow out of proportion to sample size.

What helps

Regularisation, early stopping, augmentation, simpler models, more data, ensembling.

Bias–Variance and Regularization



$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \mathrm{Bias}^2 + \mathrm{Var} + \sigma^2$$

- Bias^2 comes from the model assumption;
- Var from the estimation error;
- σ^2 is irreducible noise;
- Too simple: high bias (underfitting);
- Too complex: high variance (overfitting);
- Regularisation (L1/L2, dropout, early stopping) trades the two off.

Schematic: the vertical axis is relative error, used only to show the trend, not measured data.

Evaluation Metrics: Classification

	$\hat{y}=1$	$\hat{y}=0$
$y=1$	TP	FN
$y=0$	FP	TN

Precision $P=\frac{TP}{TP+FP}$, recall $R=\frac{TP}{TP+FN}$, $F_1=\frac{2PR}{P+R}$.

- **ROC-AUC**: ranking ability, insensitive to the threshold;
- **PR-AUC**: more sensitive and more useful when the positive class is rare;
- **Threshold choice**: driven by business cost (miss versus false alarm);
- **Calibration**: are the predicted probabilities trustworthy (essential in credit and health care);
- **Stratified reporting**: metric differences across groups (fairness).

Never report a single number: give the metric, the threshold, a confidence interval and the cost assumptions.

Evaluation Metrics: Regression and LLM Evaluation

Regression

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2}$$
$$\mathrm{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i|$$
$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

RMSE is sensitive to large errors; MAE is robust; R^2 is explained power relative to a baseline.

LLM evaluation (new)

- Task accuracy / pass rate (incl. code execution);
- Factuality: evidence, traceability;
- Robustness: paraphrase, adversarial, long context;
- Hallucination and refusal rate;
- Cost and latency (tokens, response time);
- Human review plus automated review.

Source: Classification and regression metrics follow standard statistics and machine-learning texts; LLM evaluation dimensions from 2025–2026 evaluation practice and the debate on evaluation reliability.

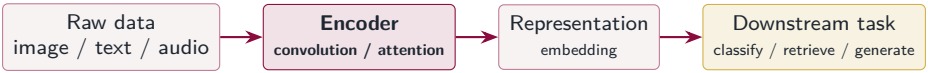
From Feature Engineering to Representation Learning

Hand-crafted features (classic)

- Depends on domain knowledge: colour histograms, *n*-grams, technical indicators;
- Interpretable, needs little data;
- Its ceiling is the designer's imagination.

Learned features (modern)

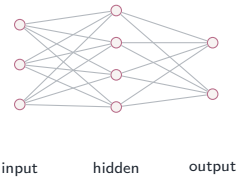
- The model learns representations straight from raw data: image, speech, text embeddings;
- Needs large amounts of data and compute;
- Representations transfer: pretrain → fine-tune or prompt.



Deep Learning ⊆ Representation Learning ⊆ Machine Learning ⊆ AI.

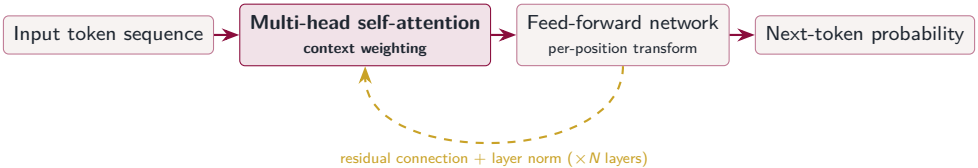
Deep Learning and Backpropagation

- Several layers of non-linear transforms: $h^{(l)} = \phi(W^{(l)} h^{(l-1)} + b^{(l)})$;
- Gradients of the loss with respect to the parameters are computed efficiently by backpropagation (the chain rule);
- Optimisation: stochastic gradient descent and its variants (momentum, AdamW);
- Engineering points: initialisation, normalisation, learning-rate schedules, mixed precision, parallel training;
- Hardware: GPU/TPU clusters with fast interconnect; training is limited by memory and bandwidth.



Source: Standard presentation of neural networks and backpropagation.

Transformers and Foundation Models

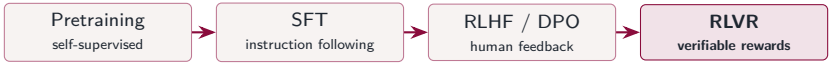


Pretraining: self-supervised "predict the next token" on a huge text corpus, which yields general representations of language and knowledge.

Adaptation: prompting, retrieval augmentation (RAG), fine-tuning (SFT/LoRA), tool calling. Context windows have grown from a few thousand to hundreds of thousands, even a million, tokens.

Source: Transformer architecture (Vaswani et al., 2017) and the standard account of the foundation-model paradigm; context-length magnitudes from public model specifications, 2024–2026.

Post-training and Alignment



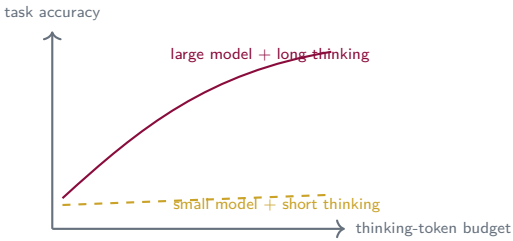
Post-training determines the model's *behaviour*: whether it follows instructions, whether it is safe, whether it is willing to reason over several steps. RLVR (reinforcement learning with verifiable rewards) is the key lever behind the reasoning gains of 2024–2026.

Capability from pretraining, behaviour from post-training; verification makes reward signals scalable.

Source: Post-training pipeline (SFT / RLHF / DPO / RLVR) compiled from public technical reports, 2023–2026.

Reasoning Models and Test-time Compute

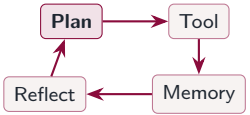
- The classic route: one forward pass produces the answer;
- Reasoning models: first generate "thinking" (a chain of thought), then the conclusion;
- Spend more inference compute to buy higher accuracy (test-time compute scaling);
- Examples: the o-series, GPT-5, Gemini Deep Think, the DeepSeek-R series;
- The price is latency and token cost, so the "thinking budget" has to be chosen per task.



Schematic: the curve is a qualitative trend, not measured data from a specific model.

Source: Reasoning-model paradigm (OpenAI o-series and GPT-5; the DeepSeek-R series; Gemini Deep Think) from public sources, 2024–2026.

Agentic AI and Tool Protocols (MCP)



Tool protocols (e.g. MCP): one interface to databases / search / code / business APIs

- An agent = model + loop control + tools + memory;
- Its ceiling depends on tool quality and permission governance;
- Key engineering issues: timeouts, retries, idempotence, audit, cost caps;
- Multi-agent collaboration: division of labour plus review (a reviewer role).

Source: Agentic AI and tool protocols (MCP) compiled from public industry sources, 2025–2026.

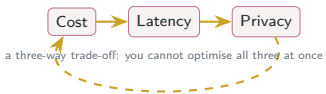
Small Models and On-device AI

Why small models matter

- Cost: at high volume the marginal cost decides feasibility;
- Latency: on-device inference skips the network round trip;
- Privacy: data never leaves the device.

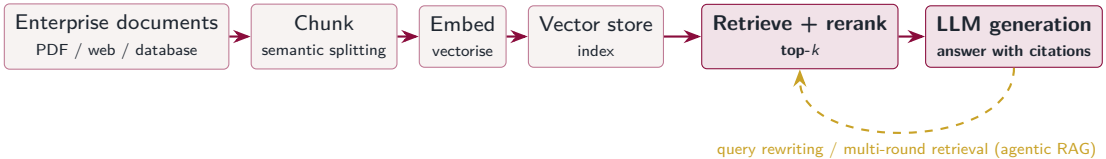
How it is done

- Distillation, pruning, quantisation (int8 / 4-bit);
- Fewer parameters plus long thinking or tool use;
- Hybrid: edge for high-frequency tasks, cloud as fallback.



Source: On-device and small-model trends compiled from 2026 reporting on edge AI and model efficiency.

RAG: Bringing Enterprise Data into the Model



What it does: it turns "memory in the parameters" into retrievable external memory, which sharply reduces hallucination and supports citations. The hard parts: chunk granularity, retrieval recall, context noise, permission filtering.

RAG is the practical bridge between a foundation model and governed enterprise data.

Source: Standard retrieval-augmented generation pipeline, compiled from post-2020 research and 2025–2026 engineering practice.

Capabilities and Limits

What models already do well

- Summarising, rewriting, translating, coding;
- Structured extraction, classification, retrieval;
- Multi-step tasks given explicit tools and verification.

Still weak or unreliable

- Factuality: confident fabrication;
- Causal and counterfactual reasoning;
- Long-horizon planning and consistency;
- Exact numerics, units, temporal reasoning.

Engineering conclusion

Treat the model as a probabilistic component: design verification around it (rule checks, unit tests, spot checks), limit permissions and keep logs.

Timeline: from AlexNet to Agentic AI (2012–2026)



Three strands intertwine: scale (data, parameters, compute), paradigm (supervised → self-supervised → post-training alignment), and form (chat → reasoning → agents).

Each wave changed what practitioners must know, not just what models can do.

Source: Milestones compiled from public technical reports and release notes; state of the art as of 2026.

2023: Multimodal Beginnings and the Open-weight Wave

Closed frontier

- Chat models entered production: long-document question answering and coding assistance became widespread;
- Multimodal ability appeared (image input);
- Firms moved from "experimenting" to "looking for use cases".

Open ecosystem

- Open-weight models appeared and iterated fast (Llama 2, Qwen, Mistral);
- Open inference and deployment toolchains matured (quantisation, local serving);
- Chinese-language and multilingual ability improved markedly.

What it meant for analytics work: text extraction, summarisation and code generation started entering everyday data workflows.

2024: Multimodality, Long Context and the Start of the Reasoning Paradigm

- **Multimodality**: unified modelling of text plus image plus audio/video became the mainstream direction;
- **Long context**: document-level and even book-level input changed the design trade-offs in RAG and retrieval;
- **Reasoning paradigm**: the o1 series turned "think first, then answer" (chain of thought plus test-time compute) into a product capability;
- **Open weights catching up**: Llama 3, Qwen2.5 and DeepSeek-V3 approached the closed frontier on many tasks;
- **Engineering**: prompt engineering → structured output (JSON schema) → tool calling → evaluation systems.

What it means for data analysis

Analysis expanded from "writing SQL or ECharts" to "driving data pipelines and chart generation in natural language". But the result must stay verifiable (numeric checks, consistent definitions, reproducibility).

First Half of 2025: the Reasoning Race and the Year of Agents

- **Reasoning race**: DeepSeek-R1 (open weights, a reproducible reasoning training recipe), o3 and the Claude 3.7/4 series followed one another;
- **Falling cost**: the price of a given capability dropped fast, and inference cost became a core selection criterion;
- **The year of agents**: the shift from "chatting" to "completing tasks" – calling browsers, terminals and office tools;
- **Standard tool protocols**: MCP and similar protocols gave models one way to reach external tools and data sources.

Why this half-year mattered

- It showed that open weights can approach the frontier;
- It showed that test-time compute can systematically buy accuracy;
- It turned a "model capability" problem into a systems and process problem.

Source: Open-weight reasoning models such as DeepSeek-R1 and industry analysis, compiled from public sources, 2025.

Second Half of 2025: New General Models and Agentic Products

- **GPT-5**: further gains in general reasoning and coding, becoming one of the enterprise baselines (OpenAI release notes);
- **Claude Opus 4.5** (Nov 2025): agentic ability for coding and long tasks became the focus of competition (industry analysis);
- **The Gemini 3 series**: multimodal reasoning and a "deep think" mode (model documentation);
- **Agentic products**: coding assistants, office assistants and data-analysis assistants began delivering whole tasks end to end;
- **Evaluation and safety**: model cards, red-teaming and enterprise-grade permissions and audit became procurement requirements.

The shift students should notice

The competitive question moved from "whose model is stronger" to "who can wire a model into a real process and keep it reliable and compliant".

2026: Open-weight Convergence and Capability Commoditisation

- **A dense run of open-weight releases**: in the first half of 2026 several labs released near-frontier open-weight models within weeks of each other, under licences that split into permissive and restrictive camps (report);
- **Capability convergence**: the gap between leading models on general tasks has narrowed, so the choice depends more on cost, latency, ecosystem and compliance;
- **Falling prices**: unit inference prices keep dropping, pushing out the boundary of what is feasible.

Open weights $\neq$ open data

- "Open weights" gives you the parameters, usually not the training data or the full training recipe;
- Licences may restrict commercial scale, redistribution or distillation;
- Check the licence terms and compliance requirements before you use a model.

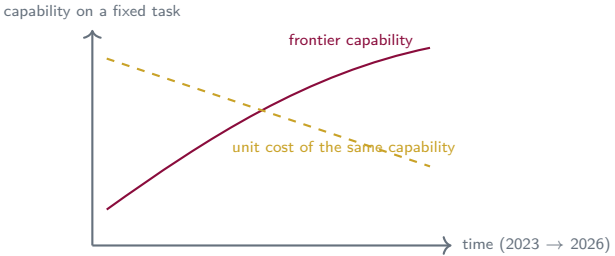
Source: Open-weight convergence and licensing models compiled from public reporting, 2026; the official licence of each model governs.

Model Comparison (as of 2026, positioning and scale)

Model	Organisation	Positioning and modality	Access / licence
GPT-5	OpenAI	General reasoning, coding, tool calling; multimodal	Closed API / product
Gemini 3 series	Google DeepMind	Multimodal reasoning, long documents, deep think	Closed API / product
Claude Opus 4.5	Anthropic	Long-horizon agents, coding, documents	Closed API / product
DeepSeek R / V series	DeepSeek	Reasoning and general dialogue; cost-sensitive settings	Open weights + API
Qwen series	Alibaba	Chinese and multimodal, full range of sizes (including on-device)	Open weights + API
Llama series	Meta	General open base for self-hosting	Open weights (community licence)
Mistral series	Mistral AI	European compliance, lightweight and efficient inference	Partly open weights

Source: Positioning comparison only; specifications and licences change with each version, so consult the official documentation before use.

Capability and Cost: the Same Capability Keeps Getting Cheaper



Choose a model by assessing task, cost and latency together, not by reading a leaderboard.

Cost structure of an LLM call

- Input tokens (context, retrieved passages);
- Output tokens (including "thinking" tokens);
- Prompt cache hit rate;
- Retries, tool calls, reruns on failure;
- Self-hosting: GPU depreciation + power + operations.

Schematic: the curve is a qualitative trend; for absolute cost and capability figures rely on official pricing and measurements.

Compute and Capex

≈$745B

AI capital expenditure by hyperscalers (2026 reporting)

≈$2.4T

infrastructure spending commitments by data-centre operators (reporting)

GW scale

electricity demand of a single AI data-centre campus

Compute has become a binding constraint on analytical capability: training sets the ceiling, inference sets the cost. Between 2024 and 2026 capital expenditure expanded from chips to power, cooling, siting and grid connection.

Compute constraints now shape business decisions: which model, which latency, which unit cost – not just which algorithm.

Source: Orders of magnitude for capital expenditure and infrastructure spending compiled from public reporting, 2026 (https://www.edgen.tech/es/news/post/hyperscalers-745b-ai-capex-fuels-semiconductor-supply-chain-demand; https://www.kucoin.com/news/flash/top-4-data-center-operators-commit-2-4-trillion-in-ai-infrastructure-spending).

Data Centres and Power

Engineering and business constraints

- Grid-connection queues and network upgrade cycles are now the first siting factor;
- High power density per rack $\rightarrow$ liquid cooling and new thermal designs;
- Water, land and community relations enter project appraisal;
- Green power and carbon disclosure affect financing and compliance.

What it implies for analytics

- Using a large model carries real environmental and financial cost;
- "Solving the problem with a small model" is an engineering virtue and cost discipline;
- Put energy and carbon into the appraisal metrics (ESG reporting needs them).

Source: Data-centre design, power and cooling trends compiled from 2026 industry reporting.

Data, Copyright and Synthetic Data

Training data

- Sources: web pages, books, code, conversations, synthetic data;
- Copyright and licensing disputes keep evolving;
- Data purchase and licensing are now a cost line.

Synthetic data

- Uses: scarce labels, privacy, hard cases;
- Risks: self-reinforcing bias, distorted distributions;
- Advice: always cross-validate against real data.

For your own project

State the data source and its licence → record how the data was processed → declare the data boundaries and limitations in the report.

The Evaluation Crisis: Leaderboards Are Not Real Capability

- **Benchmark saturation**: mainstream leaderboards are filled up, so discrimination collapses;
- **Data contamination**: test-set content may appear in training data, inflating scores;
- **Structural fabrication**: research argues current evaluations struggle to detect fabrication at the structural level (2026 study);
- **An epistemological problem in evaluation**: abilities beyond the metric (reliability, robustness) are hard to measure (2026 study).

What to do in practice

- Build your own domain evaluation set (real business samples plus edge cases);
- Report a distribution rather than a single score (including failures);
- Bring in human review and execution checks (run the code, verify the numbers);
- Record model version and prompt version so results can be reproduced.

Source: Discussion of evaluation reliability compiled from public research, 2026.

Hallucination and Mitigation

Why models hallucinate

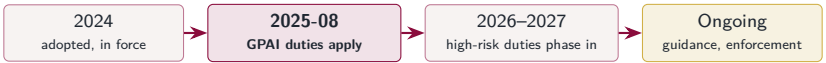
- A language model optimises the next token, not the truth of a statement;
- Training data contains errors and outdated information;
- Without external verification the model produces what "looks reasonable".

Engineering mitigations

- Retrieval augmentation (RAG) plus mandatory citations;
- Structured output plus schema validation;
- Cross-checking between models, a "reviewer" agent;
- Confidence thresholds plus human review;
- An explicit refusal policy (better to omit than be wrong).

Rule: every number and conclusion that enters a decision must be traceable to a verifiable source.

Regulation (1): the EU AI Act Timeline



The duties on firms concentrate on: risk classification, transparency (informing users, labelling content), technical documentation, data governance, human oversight and post-market monitoring.

Risk-based obligations: the higher the risk, the heavier the documentation, testing and human-oversight duties.

Source: EU AI Act (Regulation (EU) 2024/1689) timeline and obligations, compiled from official and legal-practice commentary: https://intelligence.dlapiper.com/artificial-intelligence/?t=01-law&c=EU.

Regulation (2): China's AIGC Labelling and a Compliance Checklist

- Rules on generative AI services and content labelling keep developing: the labelling measures were issued in March 2025 and took effect on 1 September 2025 (explicit labels plus implicit metadata labels) (commentary);
- The difference from the EU lies mainly in the scope of labelling, who carries the duty, and metadata requirements (comparison);
- Firms going abroad must satisfy several regimes at once.

Project compliance checklist

- Lawful, traceable data sources;
- Minimisation and masking of personal data;
- Generated content labelled as required;
- Model and prompt versions archived;
- Human review and a complaints channel;
- Cross-border data-flow assessment.

Source: Labelling requirements and China–EU differences compiled from 2025 legal commentary; the latest official documents govern.

Enterprise Adoption and ROI

- Enterprise adoption of agentic AI accelerated markedly in 2025–2026 and measurable ROI began to appear (efficiency, cost, cycle time) (report);
- What leading firms do: move from "pilot use cases" to "process redesign", with governance and evaluation alongside;
- Failure modes: no clear metric, no data foundation, no permission governance, demos that are never wired into the system.

Four common sources of gain

- Documents and knowledge work: extraction, summarisation, retrieval;
- Software development: code generation, testing, review;
- Customer interaction: service, marketing content, lead screening;
- Decision support: anomaly detection, risk alerts, scenario analysis.

Source: Evidence on enterprise adoption and ROI compiled from industry surveys and reporting, 2025–2026.

The Labour Market: Gains Are Real but Uneven

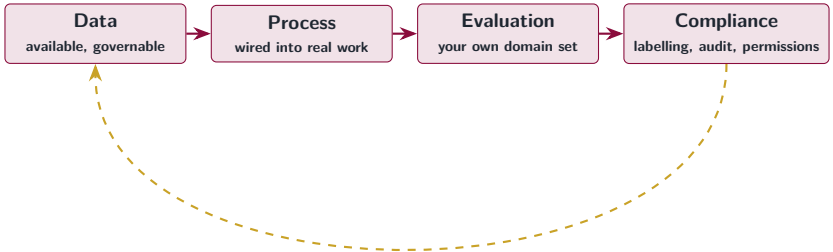
- A survey of corporate executives (March 2026) found most firms plan to expand AI use further, focused on task-level substitution and augmentation rather than whole-job cuts (Atlanta Fed);
- Research evidence: the gains from AI are large, rising and unevenly shared (`https://cepr.org/index.php/voxeu/columns/ais-gains-are-large-and-rising-unevenly-shared`, CEPR);
- The most affected tasks are clerical processing, junior coding, routine customer service and content production.

What this implies for skills

- What is scarce is people who can define the problem, verify the result and build the system;
- Data engineering, evaluation, compliance and domain knowledge are the moat;
- Writing prompts is not the core skill; designing a verification process is.

Source: Corporate surveys and research on the distribution of gains compiled from public sources, 2026.

Part 3 Summary: from Model Race to Value Loop



2023–2024: capability surprises; 2025: reasoning and agents; 2026: commoditisation + falling costs + governance duties. Competition moved from "who has a model" to "who delivers value reliably".

Model capability is increasingly a commodity; engineering, evaluation and governance are where the value is captured.

Source: Sources for this section are in the footnotes on slides 35–51.

The Data Analytics Toolbox, 2026

Foundations and analysis

- Spreadsheets (Excel / Sheets) plus HTML reports;
- Python: pandas, NumPy, Polars, DuckDB;
- SQL (cloud warehouse / local analytics);
- Notebooks (Jupyter) and VS Code.

Engineering and AI

- Version control: Git (data and code together);
- Orchestration: Airflow, dbt, Spark, Flink;
- Machine learning: scikit-learn, XGBoost, PyTorch;
- LLM toolchain: APIs, orchestration, vector stores, evals, guardrails.

Suggested order: start with the simplest tool that solves the problem (SQL or a spreadsheet), and only bring in engineering components once scale, reproducibility or collaboration becomes the bottleneck.

Languages and Tools

Tool	Strengths	Limitations	Typical use
Python	Widest ecosystem (data analysis + ML + LLM), huge community	Native speed is ordinary (made up for by vectorised and compiled libraries)	Data cleaning, modelling, automation, LLM applications
R	Very strong statistics and visualisation, mature academic ecosystem	Weaker engineering and production deployment	Statistical analysis, econometrics, chart reports
SQL	Declarative, well optimised in execution, close to the data	Awkward for complex algorithms and iterative modelling	Data extraction, aggregation, feature construction
Julia	Numerical performance close to C, modern syntax	Smaller ecosystem and talent pool	High-performance numerics, simulation
MATLAB	Complete toolboxes, teaching friendly	Commercial licence cost, closed ecosystem	Signal processing, control systems, teaching

The realistic answer is usually a combination: SQL for data + Python for modelling + Git for collaboration + an LLM for speed.

Source: Positioning summary for teaching, not a performance benchmark.

Visualization and Business Intelligence

- **BI platforms**: Tableau, Power BI – self-service analysis, dashboards, permissions;
- **Charting in code**: matplotlib / seaborn / plotly / ggplot2;
- **Interaction**: Streamlit / Shiny / Dash for quick internal tools;
- **AI assistance**: text-to-SQL / text-to-chart and automatic insight.

Charting principles (they matter more than the tool)

- Fix the question, then pick the chart type;
- Avoid dual axes and 3D; start at zero when comparing magnitudes;
- Label the sample size and the uncertainty;
- One chart, one conclusion: write the title as the conclusion.

Source: Charting principles compiled from general visualisation and data-journalism practice.

Data Engineering and Cloud

- **Batch**: Spark / cloud warehouses (columnar storage plus vectorised execution);
- **Streaming**: Kafka + Flink (real-time features, real-time risk control);
- **Transformation and orchestration**: dbt (SQL modelling plus tests), Airflow (dependency scheduling);
- **Storage**: object storage plus open table formats (Iceberg/Delta/Hudi).

Cost control in the cloud

- Partitioning and file size (small files are the enemy);
- Hot/cold tiering, column pruning, result caching;
- Reserved instances versus on-demand versus spot;
- Treat cost as an engineering metric (cost per query / per model run).

Data engineering decides whether analysis can scale and be reproduced; the model is usually the easiest part of a project to replace.

Machine Learning and Deep Learning Libraries

Classic machine learning

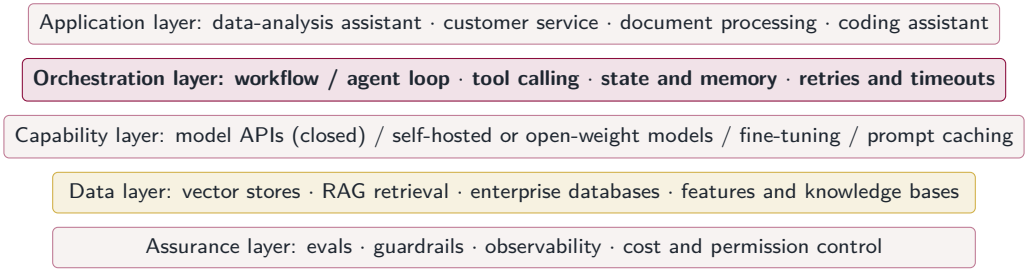
- **scikit-learn**: uniform API, pipelines, cross-validation;
- **XGBoost / LightGBM / CatBoost**: strong tabular baselines;
- Feature engineering and model selection (ColumnTransformer, GridSearchCV).

Deep learning and foundation models

- **PyTorch**: mainstream in research and industry;
- **Hugging Face**: one entry point for models, data, evaluation;
- **Fine-tuning**: LoRA/QLoRA and other parameter-efficient methods;
- **Serving**: batch, streaming, GPU sharing and queueing.

Get a tabular baseline working first (say LightGBM), then decide whether you need a deep model or an LLM.

The LLM Application Stack



The difference from the classic ML stack: three extra concerns – prompt and context management, tool and agent orchestration, and online evaluation and guardrails.

Everything that makes an LLM product reliable lives in the bottom two layers.

Reproducibility and Engineering Hygiene

- **Version control**: code, configuration, prompts and the data-splitting scheme all go into Git;
- **Environment**: pin dependency versions (requirements or lock files), containerise;
- **Randomness**: fix random seeds and record hardware and library versions;
- **Experiment tracking**: keep parameters, metrics and artefacts together (MLflow / W&B style tools).

What to record specifically for LLMs

- Model name + version + date of the call;
- The full prompt and the system prompt;
- Temperature, max tokens, tool list;
- Cost and latency statistics;
- Evaluation-set version and pass rate.

Reproducibility = someone else (including you, three months later) can rerun it and reach the same conclusion.

Cost and Compute Choices

Route	Where it fits	Main risk / cost
Cloud API (closed models)	Fast validation, peak elasticity, no operations	High unit price, data-egress compliance, vendor dependence
Self-hosted inference on cloud GPUs	Steady high volume, need for fine-tuning or private deployment	Operational complexity, expensive when utilisation is low
Local / on-device models	Privacy-sensitive, low latency, offline	Limited capability, needs quantisation and engineering effort
Self-hosted open weights	Data control, predictable cost	Licence and compliance, staffing effort

Decision rule, in plain words: work out the marginal cost per call and the compliance boundary first, then choose the technical route. Batch processing, caching, distillation and replacing with a smaller model are the three most common ways to cut cost.

Assignment 1: Tool Setup and Data Exploration

Step 1: environment (finish after this lecture)

- A Python environment (Anaconda, or uv/venv) plus Jupyter / VS Code;
- Install and run: pandas, NumPy, scikit-learn, matplotlib; optionally Polars / DuckDB;
- Sign up for and try any LLM assistant (record the model name and version).

Step 2: data exploration task (individual)

- Choose a public dataset (10,000–100,000 rows is a good range);
- Deliver: a data profile (missingness, outliers, distributions), at least three charts, and one baseline model;
- The report must include an AI use declaration (tools, prompts, what you verified and changed).

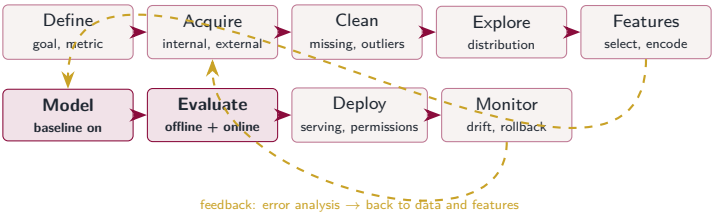
Source: Marking scheme and submission format are on a later slide (slide 80) and on the course platform.

Part 4 Summary

- Tool choice follows problem complexity: spreadsheet/SQL → Python → data engineering → LLM application stack;
- The strong baseline on tabular data is usually a gradient-boosted tree, so there is no need to start with a large model;
- The reliability of an LLM application comes mainly from three layers: evaluation, guardrails, observability;
- Cost and compliance are hard constraints on the choice, and marginal cost decides whether something can scale;
- Engineering habits (versioning, logging, experiment tracking) decide whether a project can be reproduced and trusted.

Tools change; method and the ability to verify do not go out of date.

End-to-end Workflow



The earlier a stage sits, the more it sets the ceiling: define the problem wrongly and nothing downstream can rescue it.

In practice you will loop: modeling reveals data problems, monitoring reveals design problems.

Exploratory Data Analysis

- **Univariate**: distribution shape, centre and spread, long tails, missingness;
- **Bivariate**: scatter plots, grouped box plots, correlation (incl. rank);
- **Multivariate**: correlation matrices, PCA, clustering overview;
- **Time**: trend, seasonality, structural breaks.

Four basic charts and what they give you

- Histogram / density – the distribution;
- Box plot – group differences and outliers;
- Scatter plot – relationships and non-linearity;
- Heat map – correlation and missingness structure.

The output of EDA is not "charts" but a list of testable hypotheses plus data-quality conclusions and modelling advice.

Feature Selection and Feature Engineering

Feature selection

- Filter: variance, chi-square, mutual information, correlation;
- Wrapper: recursive feature elimination (RFE);
- Embedded: L1/L2 regularisation, tree importance;
- Careful: selection must happen inside the training set, otherwise it leaks.

Feature engineering

- Numeric: log or power transforms, binning, standardisation, missing indicators;
- Categorical: one-hot, target encoding (cross-fitted), embeddings;
- Time: lags, rolling windows, cyclical encoding;
- Text: TF-IDF, *n*-grams, sentence vectors;
- Interactions and domain features (often the biggest gain).

Experience: on tabular data, feature quality usually matters more than the choice of model.

Text Data Exploration

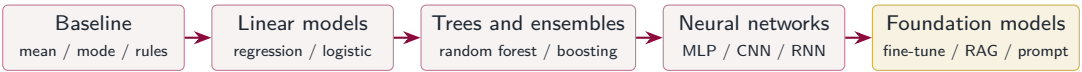
- **Splitting and normalisation**: tokenisation (different for Chinese and English), lower-casing, stop-word removal (use with care);
- **Word frequency and comparison**: TF, TF-IDF, differences in word frequency between classes;
- **Topics**: LDA / NMF / neural topic models;
- **Representations**: sentence or document vectors for clustering, retrieval and classification;
- **LLM assistance**: zero-shot labelling, summarisation, entity extraction (needs sampled verification).

Risk notes

- Stop words can carry the key sentiment ("not", "no");
- Domain vocabularies and abbreviations must be confirmed by a human;
- When an LLM labels your data, hand-label 100–200 items first as a control and report the agreement rate.

Source: Text processing follows classic text-mining practice; the advice on verifying LLM labelling is drawn from recent evaluation experience.

The Modelling Ladder



The ordering rule: simple before complex. Each step up must be shown to be better than the step below and worth its cost.

Lecture 6 derives the mathematics of linear regression and logistic regression (linear classifiers) in detail.

How to choose

Sample size (small samples favour linear or regularised models), data type (tabular → trees; text and images → neural or foundation models), interpretability requirements, latency and cost budget, and the ability to maintain the result.

Cross-validation and Hyper-parameter Tuning

- k-fold cross-validation ($k = 5$ or 10); stratified folds for imbalanced classes;
- Time series: forward rolling splits (walk-forward); never shuffle;
- Grouped data (same user or patient): GroupKFold prevents leakage;
- Nested cross-validation: tune inside, evaluate outside (avoids optimistic bias).

Tuning strategy

- Random search beats grid search in high dimensions;
- Bayesian optimisation suits expensive models;
- Early stopping plus a learning-rate schedule;
- Log every trial to avoid "manual overfitting".

Report the uncertainty: give the spread across folds or a confidence interval, not one best score.

Model Interpretation

- **Global**: coefficient sign and magnitude, feature importance, PDP/ICE;
- **Local**: SHAP values explain a single prediction;
- **Surrogate models**: approximate a complex model with a simple one;
- **Counterfactuals**: which features would have to change for the conclusion to change.

New interpretability problems in the LLM era

- Generated results cannot be explained by coefficients;
- All you can give is an evidence chain: sources, reasoning steps, tool calls;
- Hence systems designed to be traceable, reproducible and auditable;
- In high-risk settings (credit, health, justice) prefer interpretable models.

Source: Interpretation methods (PDP/ICE, SHAP, surrogate models) compiled from general interpretable-machine-learning practice.

Deployment and Monitoring

Forms of deployment

- Batch scoring (daily or hourly);
- Online API (low latency, rate limits, degradation);
- Edge or on-device (privacy and latency first);
- Human-machine collaboration (model suggests, a person decides).

What to monitor

- Input drift (PSI / KL) and feature missing rates;
- Prediction distribution and business metrics;
- Performance decay and a queue of error cases;
- Cost, latency and timeout rate;
- Model and prompt versions, rollback log.

Three questions to answer before launch

(1) Who is affected when it fails? (2) How do we roll back within an hour? (3) Who is responsible for periodically reviewing it?

One Workflow, Two Problems: House Prices and Spam E-mail

Stage	Regression task (house prices)	Classification task (e-mail)
Define the problem	Predict a continuous price; error cost is linear	Decide whether it is spam; a miss and a false alarm cost differently
Data and profiling	Floor area, age, location; missing values and outliers	Text plus sender features; classes are imbalanced
Features	Numeric binning, geocoding, interactions	TF-IDF, embeddings, behavioural features
Baseline	Predict the mean	Predict the majority class
Model	Linear regression $\rightarrow$ gradient boosting	Logistic regression $\rightarrow$ gradient boosting
Evaluation	RMSE / MAE / R^2 (split by time)	PR-AUC / precision-recall (by threshold)
Deployment	Batch valuation plus online lookup	Real-time filtering plus a human review queue
Monitoring	Price drift, error distribution	Change in spam tactics, misclassification rate

One workflow, different losses and metrics – this is precisely "Machine Learning 1-2-3" made concrete.

Evaluation and Guardrails (LLMOps)

Offline evaluation

- Build your own evaluation set: real samples + edge cases + adversarial samples;
- Metrics: accuracy, factuality, format-compliance rate, refusal rate, cost, latency;
- Regression testing: rerun after any prompt or model upgrade.

Online guardrails

- Input filtering (prompt injection, sensitive information);
- Output validation (JSON schema, numeric ranges, does the citation exist);
- Permission and tool allow-lists, call quotas;
- Logging and audit, plus an escalation path to a human.

An LLM application without evaluation and guardrails is, in essence, untested software.

Checklist of Failure Modes

- Reusing the test set to select models (hidden overfitting);
- Preprocessing before the split (leakage);
- Random splits on time-series data (future information leaks in);
- Reporting only the mean, never the distribution or the failure cases;
- Ignoring the baseline ("a complex model beats the mean" is no achievement).
- Calling a large model with no cost budget (the bill runs away);
- Writing LLM output straight into a database or publishing it;
- Depending on a single vendor and a single model version;
- Going live with no monitoring or rollback;
- Undocumented data and models (impossible to hand over or reproduce).

Three practical rules

(1) Build the baseline and the evaluation set first; (2) leave a trace for every change and keep it reversible; (3) keep a human in the loop for high-risk decisions.

Part 5 Summary

- The order and the boundaries of the workflow matter more than any single algorithm (split first, select later);
- What EDA produces is hypotheses and quality conclusions, not pretty charts;
- Feature engineering and data quality often decide the fate of a tabular project;
- Evaluation must report the distribution, the cost and the failure cases; an LLM application must have evaluation and guardrails;
- Deployment is the start of decay: monitoring, drift detection and rollback are routine work.

Trustworthy analysis = a clear goal + a reproducible process + honest evaluation.

Case Map (preview): Business Analytics Cases B1–B15

Theme	Cases (developed one by one in later lectures)
Market intelligence	B1 financial knowledge graph · B2 ESG performance · B3 event association and impact prediction · B4 alternative-data alpha
Advanced KYC	B5 customer profiling (personality, needs, value) · B6 customer behaviour prediction
Investment strategy	B7 AI trader (FX) · B8 AI trader (equities) · B9 automatic dynamic asset allocation
Customer interaction	B10 customer communication strategy · B11 insurance product sales and marketing strategy
Marketing automation	B12 automatic story generation · B13 competitive market analysis · B14 sales-lead mining
Human capital	B15 human-capital growth recommendations

This lecture gives only the map and the entry points: every case uses the same three things – data preparation, model choice, and evaluation and compliance.

Course Map



This lecture is the map: every later lecture returns to today's frame – define the problem, data, model, evaluation, deployment.

Prerequisites for the final project: data exploration (L1–L3), a predictive model (L6–L8), and an honest evaluation (all lectures).

Discussion

1. **Will AI replace data analysts?** Argue from *tasks* rather than job titles: which tasks are easiest to substitute, and which become more valuable instead?
2. Under the three constraints of **data, compute and regulation**, what are the first three things a mid-sized firm should do? Give the priority order and your reasons.
3. If a model performs well on the test set but degrades quickly after launch, what are the three most likely causes, and how would you investigate?
4. When you use an LLM to assist your analysis, what verification steps would you design to avoid confident errors?
5. Pick an industry you know and identify one setting where "the data already exists but has not yet created value". What is the obstacle?

Source: There is no single correct answer; what is assessed is the structure of the argument and the use of evidence.

Ten Takeaways

1. The value of big data lies in the decision path, not in volume;
2. Data quality and bias are first-order problems;
3. Machine learning 1-2-3: data $\rightarrow$ model $\rightarrow$ optimisation;
4. The loss and the metric must match the business cost;
5. Splitting, validation and leakage prevention are matters of discipline;
6. A foundation model = pretraining + post-training alignment;
7. Reasoning models trade test-time compute for accuracy;
8. An agent = model + tools + memory + loop control;
9. Evaluation and guardrails decide whether an LLM application can go live;
10. Compliance, cost and reproducibility are long-run competitiveness.

From "can use the tools" to "can be held responsible for the result".

Reading List

Industry and trends

- Stanford HAI, *AI Index Report 2026*;
- McKinsey, *The State of AI* (annual survey);
- KPMG, *AI Pulse* (enterprise agents and ROI);
- 2026 reporting on the open-weight frontier and licensing models (see the footnote on slide 40).

Technical and compliance

- OpenAI GPT-5 release notes (capabilities and limitations);
- Gemini 3 / Deep Think model documentation;
- Claude Opus 4.5 industry analysis (agents and coding);
- An overview of EU AI Act duties; commentary on China's AIGC labelling measures and the China–EU comparison.

Source: Full links are in the "L1 reading and data" folder on the course platform; prefer official documents and primary research.

Assignment 1: Submission and AI Use Declaration

What to submit

- Code and a data description (reproducible);
- An analysis report (PDF, $\leq$8 pages): problem, data, method, results, limitations;
- Charts ($\geq$3, with the data source stated);
- An appendix with the AI use declaration.

Declaration template

- Tools: name / version / date;
- Purpose: code / prose / analysis / figures;
- The key prompts (at least three);
- How you verified: e.g. recomputed the statistics, checked 30 sampled results;
- What you did without AI.

Failing to declare AI use is academic misconduct; declaring proper use costs no marks and can earn them.

Next Lecture

Lectures 2–3: data sourcing, cleaning and feature engineering

- Data sources and acquisition (APIs, databases, scraping, public datasets);
- Strategies for missing values, outliers and inconsistencies;
- Feature construction and encoding, plus leakage prevention in practice;
- A full exercise taking raw data to a model-ready dataset.

Preparation

- Finish the environment setup for Assignment 1;
- Choose a public dataset (ideally with both categorical and numeric features);
- Review the basics of linear algebra and probability (preparation for L6).

Source: Pace and assignment deadlines are announced on the course platform.



Thanks!

Q&A · Data Analytics in Marketing · Lecture 1

Xieer Dai (David) · Xieer.dai@szu.edu.cn · Sifang Building South 209